

CSE234 Week 12: Concepts

Last week, we covered generic types. Generic types allow us to define functions and classes once that work for a variety of data types.

However, there may be times when the generic type cannot work for any type. Concepts provide a way to constrain the types that a generic type will accept as generic arguments to ensure that the generic type will handle that concrete type. Concepts are a C++20 feature, so you'll have to ensure that you are targeting a minimum C++ standard of C++20 when compiling.

Motivating Example

Let's consider this very basic add function:

```
template <typename T>
T add(T x, T y) {
    return x + y;
}
```

This works fine if the types we pass in are numeric types, like `int` or `double`. It will also work if we pass in `std::string`, since `std::string` implements the addition operator for

```
#include <vector>

template <typename T>
T add(T x, T y) {
    return x + y;
}

int main() {
    std::vector v1 = {1, 2, 3};
    std::vector v2 = {4, 5, 6};
    std::vector v3 = add<std::vector<int>>>(v1, v2);

    return 0;
}
```

Attempting to compile this leads to a rather confusing error message:

```
test.cpp: In instantiation of 'T add(T, T) [with T = std::vector<int,
test.cpp:12:43:   required from here
test.cpp:6:14: error: no match for 'operator+' (operand types are 'std
    6 |         return x + y;
      |                ~^~
In file included from /usr/include/c++/13/bits/stl_algobase.h:67,
               from /usr/include/c++/13/vector:62,
               from test.cpp:2:
/usr/include/c++/13/bits/stl_iterator.h:634:5: note: candidate: 'templ
    634 |     operator+(typename reverse_iterator<_Iterator>::difference
      |     ^~~~~~
/usr/include/c++/13/bits/stl_iterator.h:634:5: note:   template argume
test.cpp:6:14: note:   'std::vector<int, std::allocator<int> >' is not
    6 |         return x + y;
      |                ~^~
/usr/include/c++/13/bits/stl_iterator.h:1808:5: note: candidate: 'temp
   1808 |     operator+(typename move_iterator<_Iterator>::difference_ty
      |     ^~~~~~
/usr/include/c++/13/bits/stl_iterator.h:1808:5: note:   template argum
test.cpp:6:14: note:   'std::vector<int, std::allocator<int> >' is not
    6 |         return x + y;
      |                ~^~
```

We can define a concept called `Addable` that we can use to constrain the types that can be substituted for T:

```
#include <vector>

template <typename T>
concept Addable = requires(T x, T y) {
    x + y;
};

template <typename T>
requires Addable<T>
T add(T x, T y) {
    return x + y;
}

int main() {
    std::vector v1 = {1, 2, 3};
    std::vector v2 = {4, 5, 6};
    std::vector v3 = add<std::vector<int>>>(v1, v2);

    return 0;
}
```

Now, if we try to compile this program, we get an error message like the following:

```
test.cpp: In function 'int main()':
test.cpp:18:43: error: no matching function for call to 'add<std::vector<int>>(v1, v2);'
   18 |         std::vector v3 = add<std::vector<int>>>(v1, v2);
      |                                     ~~~~~^~~~~~
test.cpp:11:3: note: candidate: 'template<class T> requires Addable<T> T add(T x, T y) {'
   11 | T add(T x, T y) {
      |   ^~~
test.cpp:11:3: note:   template argument deduction/substitution failed:
test.cpp:11:3: note:   constraints not satisfied
test.cpp: In substitution of 'template<class T> requires Addable<T> T add(T x, T y)':
test.cpp:18:43:   required from here
test.cpp:5:9:   required for the satisfaction of 'Addable<T>' [with T = std::vector<int>]
test.cpp:5:19:   in requirements with 'T x', 'T y' [with T = std::vector<int>]
test.cpp:6:7: note: the required expression '(x + y)' is invalid
    6 |         x + y;
      |         ~~~
cc1plus: note: set '-fconcepts-diagnostics-depth=' to at least 2 for more details
```

Here, we get a clearer error message that the compilation failed because the type `std::vector<int>` does not satisfy the concept `Addable`, which requires `x + y` to be a valid expression. We can also make the code even clearer by replacing `typename` with `Addable` in `add`'s template parameter list:

```
#include <vector>

template <typename T>
concept Addable = requires(T x, T y) {
    x + y;
};

template <Addable T>
T add(T x, T y) {
    return x + y;
}

int main() {
    std::vector v1 = {1, 2, 3};
    std::vector v2 = {4, 5, 6};
    std::vector v3 = add<std::vector<int>>>(v1, v2);

    return 0;
}
```

The `concepts` library provides predefined concepts in the `std` namespaces. There is a pretty limited set of predefined concepts, and they can be used just like programmer-defined concepts, so they aren't going to be covered in depth. The list of predefined concepts is available on cppreference.com.

Concept Declarations

A concept declaration consists of a generic parameter list, followed by the `concept` keyword and a name. The name of the concept is followed by an equals sign and a "requires expression". The requires expression starts with the `requires` keyword and is followed by two possible items (when used for a concept declaration), a block, or a parameter list with a block. In most instances, the parameter list is needed since this allows you to specify what instances of the generic type can do. A block without the parameter list only allows you to express a limited number of items, such as the static members of the generic type or inner types.

```
template <typename T>
concept HasInnerType = requires {
    T::inner
};
```

In this case, class `A` will pass the concept check, and class `B` will not. Neither will class `C`.

```
class A {
public:
    A() {}
    class inner {};
};
class B {
public:
    B() {}
};
class C {
public:
    class inner1 {};
};
```

More interesting is when we add the parameter list. This allows us to

specify actions that an instance of the generic type can take:

```
template <typename T>
concept HasSize = requires(T t) {
    // verifies that T instances have a member function size
    t.size();
};

template <typename T>
concept Addable = requires(T x, T y) {
    // verifies that T has "operator+(T, T)" defined
    x + y;
};

template <typename T, typename U>
concept MixedTypeArithmetic = requires(T t, U u) {
    // verifies that T and U can be added
    t + u;
    // subtracted
    t - u;
    // multiplied
    t * u;
    // divided
    t / u;
};
```

Here are some examples of these concepts in use:

```
template <HasSize T>
size_t getSize(T object) {
    return object.size();
}

template <Addable T>
T add(T x, T y) {
    return x + y;
}

// concepts that use multiple types need a requires clause
// the concept cannot be used in place of "typename"
template <typename T, typename U>
requires MixedTypeArithmetic<T, U>
T add(T x, U y) {
    return x + y;
}
```

This type of check do not verify the return type of the expressions. To check the return type of the expression, the `{}` -> `conceptName;`

syntax is used:

```
template <typename T>
concept HasSize = requires(T t) {
    {t.size()} -> std::same_as<size_t>;
}

template <HasSize T>
size_t getSize(T object) {
    return object.size();
}

template <typename T, typename U, typename R>
concept DivisionReturnsR = requires (T t, U u) {
    {t / u} -> std::same_as<R>;
}

template <typename T, typename U, typename R>
requires DivisionReturnsR<T, U, R>
R divide(T x, U y) {
    return x / y;
}
```

Conjunctions and Disjunctions

If a generic type/function must satisfy multiple concepts, a conjunction (`&&`) can be used:

```
template <typename T>
concept Integral = requires std::is_integral<T>;

// both Integral<T> and std::is_signed<T> concepts must be met
template <typename T>
concept SignedIntegral = requires Integral<T> && std::is_signed<T>;
```

If a generic type/function must satisfy one or another concept, a disjunction (`||`) can be used in the same way.

Requires Requires

If you want use an inline concept (i.e., use a `requires` expression without defining a concept), you have to use the somewhat silly looking `requires requires` syntax. The second `requires` can have any expressions that a normal `requires` in a concept can have:

```
template <typename T>
requires requires(T x, T y) {
    x + y;
}
T add(T x, T y) {
    return x + y;
}
```

Conclusion

Concepts provide a way to make constraints on generic types clearer. The error messages for a generic type violation is more clear when a concept is used. Additionally, using concepts allows editor error detection (red squiggles) to detect when the template arguments are invalid, making it faster to identify when a generic function/type is being misused.